
TAPESTRY: AN OPERATING SYSTEM FOR PHYSICALLY RECONFIGURABLE MATTER

James V. Steele
Independent Researcher
Mountain View, CA
jim@tapestry-os.com

June 6, 2026

ABSTRACT

Collectives of coordinated physical elements that sense, compute, communicate, and act to produce emergent physical behavior are advancing rapidly across domains including aerial swarm robotics, warehouse automation, morphing aerospace structures, surgical microrobotics, and precision agriculture. Despite this hardware progress, the lack of a shared software abstraction layer between physical elements and the applications that direct collective behavior forces every research group and commercial deployment to rebuild fundamental coordination infrastructure from scratch.

Tapestry is introduced as an open-source operating system for physically reconfigurable matter that addresses this fragmentation. Tapestry’s architecture encapsulates heterogeneous hardware, communication substrates, and physical actuation mechanisms, from high-performance ARM-based silicon in today’s autonomous robots to biochemical state machines in nanoscale molecular systems, enabling application code written for one physical scale to execute on any other without modification. In addition, nine open research challenges are identified as opportunities to extend Tapestry’s capabilities and scale, spanning distributed systems theory, control theory, materials science, and synthetic biology. Tapestry is released as open source at <https://github.com/tapestry-os> under the Apache 2.0 license, providing a stable, demonstrable platform against which solutions to these challenges can be independently developed and empirically validated.

Keywords operating systems · programmable matter · swarm robotics · emergent systems · collective coordination · distributed state management · scale-invariant systems · synthetic biology · hardware abstraction layers

1 Introduction

In recent years, physically reconfigurable matter has transitioned from theory to large-scale deployment across diverse fields. Notable examples include aerial drone swarms exceeding one thousand elements [1], warehouse fleets coordinating tens of thousands of mobile robots [3], morphing aerospace structures demonstrating significant aerodynamic gains in flight tests [7], and surgical microrobots entering clinical trials for targeted drug delivery [9]. Despite these advancements, progress remains siloed; each domain relies on bespoke hardware and monolithic software stacks. Developers are forced to reinvent fundamental distributed primitives, such as decentralized state management, opportunistic mesh routing, and scale-free consensus, for every new deployment. Without an open standard to define the programming of reconfigurable matter, the field remains a landscape of incompatible, bespoke software stacks that prevent the accumulation of shared capabilities (see Table 1).

A historical parallel exists in the mobile computing industry of the early 2000s. The market was fractured by proprietary operating systems and carrier-specific middleware, requiring developers to port applications across dozens of incompatible platforms. Android resolved this by implementing an application framework and hardware abstraction model that established a stable boundary between the underlying device and the application. This architectural shift enabled an ecosystem where a single software framework could govern a vast diversity of hardware—from smartphones and wearables to industrial displays.

Table 1: Representative physically reconfigurable systems and their software gaps

System	Description	Software Gaps
<i>Near-term 1–3 yr</i>		
Industrial inspection [2]	Coordinated swarms of crawling or flying elements that inspect confined, hazardous, or inaccessible industrial infrastructure.	GPS-denied navigation needs a world model capable of integrating heterogeneous sensor streams into a coherent structural state.
Logistics automation [3]	Large coordinated fleets of mobile robots that dynamically reconfigure warehouse layouts and transport inventory.	Needs a direct collective coordination mechanism for dynamic task allocation.
Exoskeletons [4]	Wearable structures of discrete articulated elements that conform to, assist, or augment the human body.	Requires sub-10ms latency orchestration; no shared platform exists across companies despite similar architectures.
<i>Mid-term 3-7 yr</i>		
Precision agriculture [5]	Micro-drone swarms that monitor crops and apply treatments with element-level precision.	Needs an overarching state manager of the collective for real-time sensor fusion and graceful degradation under sensor loss.
Morphing aircraft [6]	Aerospace surfaces where distributed elements sense load and change shape to optimize aerodynamic efficiency.	Software platform for deterministic operating modes while preserving adaptive capability does not yet exist.
<i>Research / defense</i>		
Drone swarms [8]	Collections of simple robots producing collective behaviors without a central controller.	No production OS layer; coordination logic does not exist in a reusable deployable form.
Microrobotics [9]	Millimeter-scale elements capable of locomotion and manipulation, typically teleoperated.	No platform abstraction; every robot requires rewriting the coordination stack without reusable middleware.
<i>Long horizon 15+ yr</i>		
Molecular machines [10]	Nanoscale structures performing work or logic through molecular interactions.	Hardware abstraction requires a fundamentally new model for stochastic computation.

This paper introduces the Tapestry operating system (OS), an architecture designed to unify the fragmented landscape of physically reconfigurable matter. By applying the separation of concerns principles that have governed OS design for half a century, Tapestry defines a seven-layer architecture that decouples physical element hardware from collective application software, enabling application code written for one physical substrate to execute on any other without modification. This framework is intentionally scale-agnostic: it provides a stable application programming interface (API) for coordinating behavior across systems ranging from today’s aerial swarms to submicron molecular agents on a ten-to-twenty year horizon.

The rest of this paper is organized as follows. Section 2 surveys related work in distributed systems, programmable matter, and swarm robotics middleware. Section 3 presents the seven-layer Tapestry architecture and how each layer evolves across physical scales. Section 4 demonstrates the architecture through illustrative use cases, and Section 5 empirically validates Tapestry’s core implementation using both simulations and physical hardware. Section 6 describes Tapestry’s open source strategy. Before concluding, Section 7 identifies nine open research problems as opportunities to extend and scale Tapestry.

2 Related Work

Tapestry draws on and extends several bodies of prior work spanning distributed systems, swarm robotics, and operating systems design. Analyzing how these disparate fields approach the trade-offs of scale and abstraction establishes the context for Tapestry’s multi-layered coordination model.

2.1 Distributed State and Consistency Models

The CAP (Consistency, Availability, and Partitioning) theorem [11] establishes that distributed data storage systems must choose between consistency and availability during network partitions. Tapestry extends this framework to

states comprising physical positions and material configurations that evolve continuously and autonomously in time. Unlike abstract data values in a traditional database, a physical element’s state changes independently of network write operations; for instance, an element’s spatial position at time $T + 1$ differs from $T + 0$ due to ongoing kinetic motion. This introduces temporal validity decay as a core property of the consistency model. While Conflict-free Replicated Data Types (CRDTs) [12] provide deterministic merge semantics for eventually consistent state, they do not account for time-bounded validity or spatial indexing. Real-time database systems [13] introduced temporal validity intervals for time-sensitive data; Tapestry extends this concept to distributed, spatially indexed physical state, offering tunable consistency modes exposed as a runtime application parameter.

2.2 Programmable Matter and Ensemble Computing

Prior research into programmable matter has largely targeted specific physical scales or substrate constraints. Claytronics [20] pioneered the ensemble programming concept for macroscale reconfigurable hardware, identifying the challenge of deriving individual element operations from a single, high-level collective shape goal. At sub-millimeter scales, Smart Dust [21] established the operational limits for sensing nodes under extreme energy restrictions, directly informing Tapestry’s power-domain scheduling framework. At the nanoscale, DNA nanotechnology [22] and strand displacement computing [23] demonstrated executable computation within molecular substrates, motivating Tapestry’s requirement for biochemical abstraction. Crucially, none of these isolated efforts address the unifying systems software layer required to port a common programming model across disparate physical scales, a gap that Tapestry’s multi-layered architecture explicitly fills.

2.3 Consensus Protocols

Traditional leader-based consensus protocols, including Raft [14] and Practical Byzantine Fault Tolerance (BFT) [15], provide strong consistency guarantees for replicated state machines. These frameworks assume homogeneous message-passing networks with bounded latency, fixed participant sets, and statistically independent node failures. All three assumptions are invalid in physically embodied element swarms: communication substrates can span RF (millisecond latency), acoustic (tens of milliseconds), and chemical gradient signaling (seconds to minutes); participants join and leave continuously as elements move in and out of communication range; and failures are spatially correlated — a physical collision eliminates multiple neighboring communication links simultaneously. Delay-tolerant networking protocols [16] address heterogeneous and intermittent connectivity but assume store-and-forward semantics incompatible with real-time physical coordination. Tapestry addresses these limits by introducing a scale-adaptive architecture that shifts fluidly between explicit BFT consensus, stigmergic indirect coordination, and reaction-diffusion dynamics based on localized substrate capabilities.

2.4 Swarm Robotics Programming Models

The Buzz programming language [17] provides a swarm programming model with virtual stigmergy: a distributed key-value store propagated by gossip across robot collectives. This establishes the feasibility of gossip-based distributed state in swarm systems. Tapestry builds on this core concept by introducing a formalized consistency model for network partitions, featuring explicit partition detection via quorum analysis and a formal reconciliation mechanism. Furthermore, while Buzz’s virtual stigmergy operates as a generic key-value store lacking spatial semantics or entry expiration, Tapestry’s world model is explicitly typed for spatially indexed physical positions, utilizing staleness thresholds derived from gossip interval timing.

2.5 Multi-Robot Middleware

ROS 2 [18] is the dominant middleware for multi-robot systems, providing a distributed publish-subscribe communication model over a Data Distribution Service (DDS). ROS 2 assumes reliable Ethernet or WiFi connectivity, has a large memory footprint incompatible with resource-constrained embedded elements, and provides no formalized model for collective state consistency or partition handling. Its node-based architecture addresses communication between processes running on capable hardware; but it does not address the resource-constrained, mesh-networked, fault-tolerant collective state management that physical element collectives require. Fleet management platforms for robotic systems [19] treat individual robots as independently addressable units coordinated from a central or cloud-based system, without modeling collective state or emergent coordination. Tapestry adopts the operating system philosophy and QoS tier concept from ROS 2 and extends it to operate without reliable network infrastructure, without a central coordinator, across physical scales that ROS 2’s memory footprint precludes, and with a formalized collective state model that ROS 2 does not address.

Table 2: The Tapestry seven-layer stack.

Layer	Tapestry Name	Responsibility	OS Analogy
L7	Choreographer	Developer-facing API that turns Goals into Intents: provide directives, shape policies, behavioral objectives.	User Applications
L6	Behavior Synthesis Engine	Translates Intents into per-element behavioral directives; task decomposition; runs optimization across swarm.	Compiler / Runtime
L5	Swarm Coordination Runtime	Distributed coordination: consensus, message-free role election, and fault tolerance across elements.	Process Scheduler / IPC
L4	Collective State Manager	Maintains a distributed world model; aggregates sensor data from all elements into a coherent shared state.	Virtual Memory / Filesystem
L3	Inter-element Communication	Mesh networking, routing, bandwidth management, encryption; RF abstraction, acoustic, optical substrates.	Network Stack (TCP/IP)
L2	Element Runtime	Per-element OS: local scheduling, power management, actuation control, local sensing, hardware abstraction.	OS Kernel
L1	Physical Substrate Interface	Per-element HAL: motor drivers, sensor buses, power sources, communication transceivers.	Hardware Drivers / BSP

3 Tapestry Architecture and Scale Evolution

Tapestry’s execution model starts with a single governing principle to enable emergent behavior in collectives: push computation toward the element, and pull upward only what the element physically cannot do. Any layer that requires centralized execution limits both the collective’s fault tolerance and its deployability across physical scales. This motivates defining a set of overarching tenets for the development and evolution of Tapestry.

3.1 Tenets of Tapestry

- **Physical Scale Invariant:** Must maintain coherent collective behaviors (e.g., consensus, role election, task allocation, fault recovery) even as underlying physical dynamics and constraints shift across scales.
- **Hardware Agnostic:** Must decouple higher-layer coordination and application logic from the execution substrate, allowing it to run unchanged on platforms ranging from contemporary silicon microcontrollers to future molecular logic gates.
- **Element Independence:** Must ensure every element operates as a fully autonomous participant so the system remains resilient under arbitrary node loss, eliminating single points of failure that restrict deployable scale.
- **Designed Emergence:** Must cultivate complex collective behaviors that transcend individual programming while strictly enforcing the deterministic safety bounds required by mission-critical medical and aerospace applications.
- **Secure by Design:** Must route all inter-element communication through strictly defined, authenticated APIs, confining trust entirely to the individual element’s runtime with zero implicit trust between nodes or external systems.
- **Developer-First:** Must empower domain experts with no systems programming background to orchestrate collective physical behavior through a high-level, intent-driven, and platform-independent API.

3.2 Tapestry Stack Overview

To realize these tenets across heterogeneous environments, Tapestry abstracts the requirements of an ad-hoc collective into a structured, seven-layer stack (Table 2). Deliberately mirroring the classical OSI model, this architecture provides immediate technical intuition by applying the separation of concerns at layer boundaries, networking’s proven scaling strategy, directly to physical coordination. See Figure 1 for an illustration of the Tapestry logical flow.

Using the Software Development Kit (SDK), developers declare Goals into a cross-collective application called a "Choreo" (inspired by dance choreography, which specifies what the performance achieves, not the path of each dancer). Layer 7 (the Choreographer) resolves these Goals (the "What") via late binding into Intents (the "How"), which are functional primitives. Layer 6 (the Behavioral Synthesis Engine, or BSE) analyzes these incoming Intents

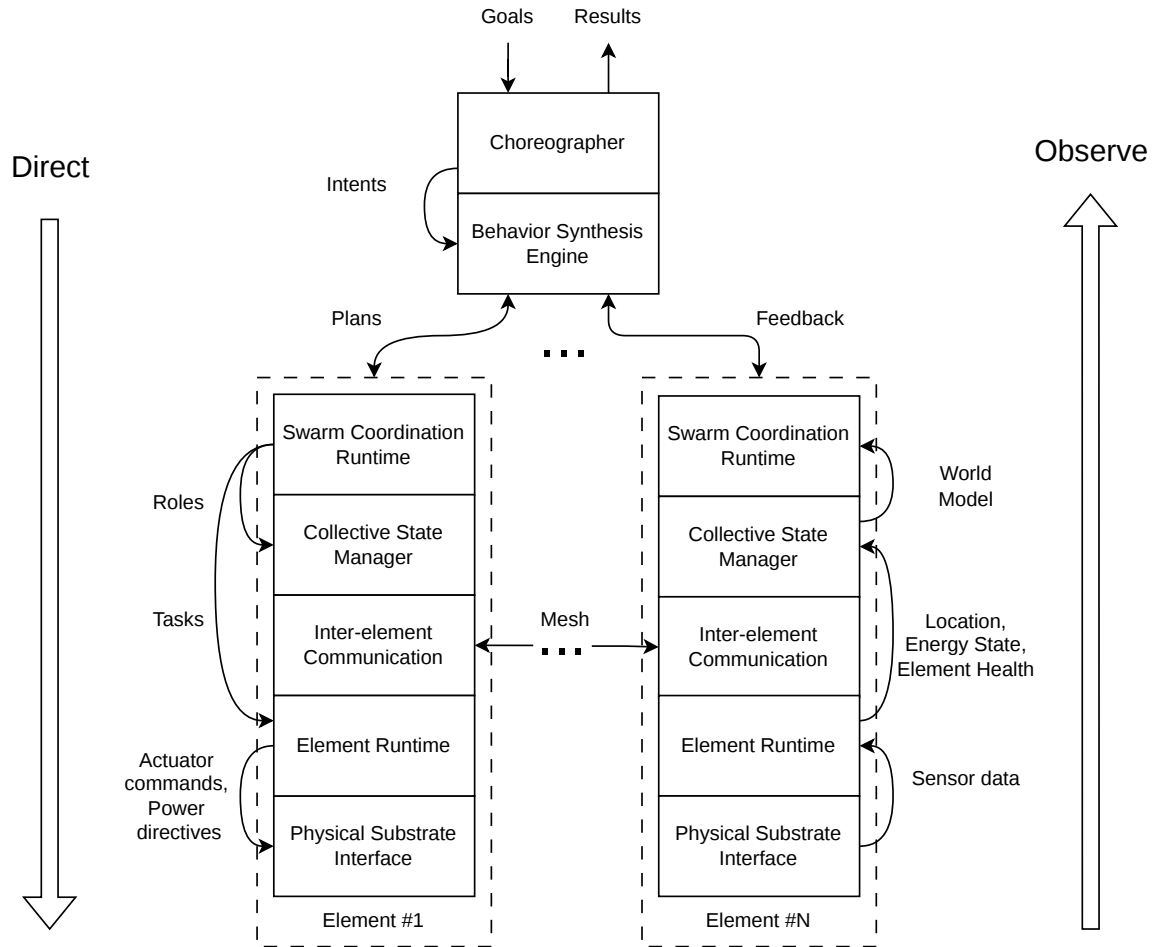


Figure 1: Tapestry stack logical data flow. L1–L5 execute on each physical element; L6–L7 execute on external compute (developer workstation, edge node, or cloud) or a sufficiently capable element within the collective. Arrows represent logical data flows; physical transmission through L3 is omitted for clarity.

and decides which ones take priority based on the current state of the immediate environment. If two different Choreos have conflicting Goals, the BSE acts as the referee.

Layer 5 (the Swarm Coordination Runtime, or SCR) synchronizes local BSE plans across the collective. By managing peer-to-peer communication and temporal alignment, it facilitates the emergent behavior necessary to transform individual actions into a unified, cohesive movement of the physical landscape. Individual elements maintain a localized view of the system via Layer 4 (the Collective State Manager, or CSM) and communicate through a Layer 3 mesh protocol. Layer 2 (Element Runtime) and Layer 1 (the Physical Substrate Interface, or PSI) are the per-element OS and hardware abstraction layer (HAL).

Tapestry’s physical-scale-invariant and hardware-agnostic tenets mandate absolute structural coherence as element dimensions shrink across multiple orders of magnitude. Table 3 summarizes how each layer of the Tapestry stack evolves across the full scale spectrum and further detailed in the following subsections. The architectural anchor of this design is the principle of asymmetric stability: the developer-facing application surface (L6-L7) remains strictly invariant, while the foundational execution layers (L1-L2) adapt heavily to the shifting physics of smaller scales.

Table 3: Tapestry layer implementations across physical scales. L1–L5 implementations evolve across all scales while L6–L7 remain stable. Application code written today should execute on future nanoscale elements without modification.

	Macro (cm) — Now	Micro (mm) — 3–8 yr	Nano (nm) — 15–20 yr
L1	Microcontrollers, MEMS sensors, BLE/UWB radio, EM actuators	Ultra-low-power ASICs, UWB, piezo actuators	Biocomputing units, chemical gradient XCVRs, molecular actuators
L2	Zephyr RTOS	Custom minimal RTOS, <10 kB	Biochem state machine; chem logic
L3	BLE mesh, UWB ranging	Acoustic, optical, near-field RF	Chemical signaling, ligand binding
L4	Distributed in-memory flat table	Compressed sparse state	External macroscale state store
L5	BFT consensus; full task allocation	Stigmergic coordination	Reaction-diffusion dynamics
L6	Edge/Cloud with physics planner	Same with on-element policy	Same with fixed-design policy
L7	Full SDK; rich developer tooling	Same	Same

3.3 L1 — Physical Substrate Interface

The PSI is the boundary between software and matter. It provides a standardized HAL over the physical substrates that Tapestry elements may be built on: silicon microcontrollers, MEMS devices, biological scaffolds, or molecular logic at nanoscale.

Key design decisions at this layer:

- Driver model inherits its structured, devicetree-based semantics from Linux: physical capabilities are declared, not hard-coded.
- Power domain management is a first-class citizen; most elements are small and battery powered.
- Actuation primitives are abstract (translate, rotate, bond, release, emit) rather than motor-specific.
- Communication transceiver abstraction covers RF, acoustic, optical, and (at nanoscale) chemical signaling.

At centimeter scale, L1 is implemented using mature embedded hardware: ARM Cortex-M series processors with MEMS inertial sensors, BLE radio transceivers, and motor driver peripherals. Adding a new hardware target requires writing a devicetree binding and driver; the layers above L1 remain unchanged. To ensure that the system foundation can be compiled onto highly resource-constrained substrates, Tapestry’s lower core layers are written in standard C99, the universal baseline for embedded systems. At millimeter scale, the power and area budgets reduce, requiring custom ultra-low-power ASICs for processing. Piezoelectric actuators replace electric motors; UWB or acoustic transceivers replace BLE radios; analog front-end circuits replace digital sensor peripherals. Again only the L1 driver implementation changes.

At nanoscale, software executing on silicon is replaced by molecular geometry and reaction kinetics of the element itself. The devicetree declaration mechanism is biological, but extends conceptually to this scale. The yet-to-be-invented L1 interface must accommodate element binding domain sequences, conformational transition rates, and chemical emission and reception capabilities (Section 7).

3.4 L2 — Element Runtime

The Element Runtime is the per-element operating system. It is intentionally minimal — its design philosophy is closer to a real-time OS (e.g., FreeRTOS, Zephyr) than to a general-purpose OS. Each element may have severely constrained compute (sub-MHz clock, kilobytes of memory), requiring careful design.

Key design decisions at this layer:

- Local task scheduler: priority-based, with deterministic interrupt latency guarantees.
- Power state machine: active, idle, deep-sleep, and harvest modes with seamless transition.
- Local perception pipeline: sensor fusion at the element level before data is propagated upward.

- Secure enclave: element identity, cryptographic attestation, tamper response. Critical for medical and infrastructure applications. OTA update subsystem: elements must be remotely updatable without physical access.

At centimeter scale, L2 can be a conventional RTOS. Elemental compute at this scale has enough RAM and clock speed to run the layers above simultaneously as well. At millimeter scale, L2 must be pared down (<10kB of flash) to a custom hardwired cooperative scheduler with a handful of task slots, a power state machine with hardware-controlled transitions, and a stripped local sensing pipeline. At nanoscale there is no software runtime at all: elemental behavior is fixed at fabrication time. The behavioral program synthesized by L6 at design time needs to be instantiated in the element's molecular geometry before the element is manufactured. Still, the L2 interface is identical.

3.5 L3 — Data Transport

This is the network stack of Tapestry. The fundamental challenge here is that conventional networking assumptions break down in element swarms: the network topology changes dynamically as elements move, power on, and fail; bandwidth is severely constrained; and latency requirements may be real-time (surgical nanobots) or relaxed (structural material monitoring).

Key design decisions at this layer:

- Mesh routing protocol: topology-aware, designed for high churn. Inspired by BATMAN (Better Approach To Mobile Adhoc Networking) [26] but optimized for dense, short-range element clusters.
- Gossip protocol layer: eventual consistency for non-critical state propagation; lower overhead than broadcast.
- Quality of Service (QoS) tiers: hard real-time (critical actuation), soft real-time (coordination), best-effort (telemetry).
- Substrate polymorphism: the same L3 API works whether the underlying transport is BLE mesh, ultrasonic acoustic channels, or chemical gradient signaling at nanoscale.
- End-to-end encryption and element authentication built in from the beginning.

At centimeter scale, L3 is implemented over BLE mesh or equivalent transport, providing multi-hop routing, gossip protocol execution, QoS management, and end-to-end encryption.

At millimeter scale, the antenna size for RF becomes physically impractical. L3 transitions to acoustic communication via piezoelectric MEMS transducers, near-field electrical coupling, or optical modulation via integrated photodetectors. These substrates operate at shorter range and lower bandwidth than RF but scale physically with the element dimensions. The L3 API remains unchanged — the gossip protocol, routing, and QoS mechanisms operate identically above the substrate abstraction.

At nanoscale, L3 must operate through release of signaling molecules that diffuse to neighboring elements [27]. This communication substrate is continuous rather than discrete, broadcast rather than addressed, and governed by diffusion physics rather than electromagnetic propagation. The latency is seconds to minutes rather than milliseconds. The bandwidth is bits per minute rather than megabits per second. Still, the L3 interface is identical (Section 7).

3.6 L4 — Collective State Manager

The CSM maintains the distributed world model: the shared understanding of the system's physical state that no single element could maintain alone. The CSM must solve the classic distributed systems problem of consistency under partition, but with the additional constraint that the "data" being managed represents physical positions, material states, and environmental conditions that evolve in continuous time.

Key design decisions at this layer:

- Spatial index: distributed octree structure partitioned across elements by physical location. Inspired by geospatial database indexing but designed for extreme scale and dynamism.
- Sensor fusion engine: Bayesian state estimation (extended Kalman filter at scale) aggregating local percepts into a coherent world model.
- Consistency model: runtime-tunable between Consistent/Partition-tolerant (CP) and Available/Partition-tolerant (AP) operational modes. CP mode freezes own-state updates on quorum loss, bounding divergence at the cost of operational continuity; AP mode maintains element operation through partition at the cost of

accumulated state error. The active mode is selectable per deployment without modifying the coordination stack.

- Memory pressure management: elements have finite memory; the CSM manages what state is cached locally and what is evicted to the collective.

At centimeter scale, each element at L4 maintains a distributed in-memory world model across all elements in RAM, gossips its authoritative state at 500ms intervals, and applies Lamport clock merge semantics to received updates. The consistency metric, CP/AP mode selector, and partition reconciliation mechanism run on every element.

At millimeter scale, reduced available RAM per element requires L4 to operate in a compressed sparse mode: each element holds only its own authoritative state and the entries for its immediate mesh neighbors, evicting distant entries aggressively to save memory. More capable elements designated by L5's role election mechanism, or nearby edge hardware, need to become regional aggregation nodes to hold world model data beyond their immediate neighborhood which individual elements can query. The L4 API remains unchanged above this tiered storage model.

At nanoscale, elements cannot hold any meaningful world model state. L4 migrates entirely to a monitoring system external to the element collective which assembles the world model from element telemetry and makes it available to L5 and L6 on external hardware. Individual elements contribute only their own raw sensing data upward; they do not participate in world model maintenance. This is the point at which the no-central-coordinator property is most significantly qualified: the external world model store is a form of centralization that the physics of nanoscale computation makes unavoidable. The architectural response is to treat this external store as infrastructure analogous to the network itself: a shared resource that the collective depends on but that is not a coordination bottleneck in the sense of a single point of failure for collective behavior.

3.7 L5 — Swarm Coordination Runtime

The SCR is the heart of Tapestry's collective intelligence. It implements the distributed algorithms that produce coherent collective behavior from simple local rules — the computational equivalent of how biology achieves emergence.

Key design decisions at this layer:

- Consensus layer: BFT consensus for decisions that must survive element failure or compromise.
- Role election: dynamic assignment of roles (leader, relay, sensor, actuator) to elements based on position, capability, and health.
- Task decomposition engine: breaks high-level objectives received from L6 into subtasks assignable to individual elements or element subgroups.
- Fault detection and reconfiguration: monitors element health, triggers role reassignment on failure, maintains system function through graceful degradation.
- Quorum management: determines minimum viable collective size for a given task; initiates abort protocols if quorum cannot be maintained.

At centimeter scale, L5 implements explicit BFT consensus. Every element participates in consensus rounds, casts votes, and verifies quorum. Role election, task allocation, and fault recovery all operate through multi-round message exchange. This regime provides the strongest coordination guarantees and is appropriate for elements with sufficient compute and communication resources to sustain the message overhead of BFT protocols.

At millimeter scale, the computational and communication resources required for explicit BFT consensus exceed what resource-constrained elements can sustain. L5 transitions to stigmergic coordination: elements coordinate indirectly by reading and modifying shared state entries in the L4 world model — analogous to pheromone trail dynamics in social insects — without direct message exchange for coordination decisions. Role election becomes a gradient convergence process; task allocation operates through competitive exclusion in shared state; fault recovery operates through regenerative gradient re-establishment. Coordination is eventually consistent rather than immediately consistent, but the L5 interface presented to L6-L7 is identical.

At nanoscale, explicit coordination of any kind is physically impossible from within the element collective. L5 transitions to reaction-diffusion dynamics: collective coordination emerges from the spatial dynamics of chemical concentration fields produced by element emission and consumption of signaling molecules [27]. Role differentiation, task claiming, and fault recovery all emerge from the physics of these chemical gradients rather than from programmed coordination logic. The formal equivalence between these three coordination regimes — BFT consensus, stigmergic coordination, and reaction-diffusion dynamics — as implementations of the same abstract L5 interface is yet to be solved (Section 7).

3.8 L6 — Behavior Synthesis Engine

The BSE is the compiler and runtime for Tapestry Choreos. It receives high-level Intents from L7 and synthesizes the behavioral plans that the SCR will execute. The BSE does not mutate SCR state. L5 remains the authoritative safety and coordination layer and calls into L6. The BSE provides the behavioral program that L5 executes within its quorum envelope.

Key design decisions at this layer:

- Intent parser: translates Goals ("form a load-bearing arch," "navigate to target tissue," "reconfigure into thermal insulator") into formal behavioral specifications.
- Physics-aware planner: constraint satisfaction over the physical laws governing element behavior at the relevant scale; what is geometrically possible given current configuration.
- ML inference runtime: cross-platform ML inference engine for on-device policy execution, supporting TFLite, ONNX, and quantized model formats.
- Simulation bridge: tight integration with a physics simulation environment for offline training and testing of Choreos before deployment on real elements.
- Feedback controller: closed-loop control between intended and actual physical state; Tapestry manages the control loop so application developers do not have to.
- Supports a prioritised goal queue with preemption and sequencing.

At centimeter scale, a full BSE runs on capable elements or an edge node / cloud instance near the collective. The physics-aware planner operates with rich aerodynamic or ground vehicle models, the emergent behavior synthesis algorithm has access to high-fidelity physics simulation, and the ML inference runtime can deploy compact quantized models to elements with sufficient compute. Behavioral programs are synthesized at runtime in response to application objectives and updated as collective state evolves.

At millimeter scale, the BSE operates in a hybrid mode: a simplified physics planner runs on an edge node, producing behavioral programs that are downloaded to elements as compact local policies. On-element policy inference executes within the element's limited compute budget while the BSE on the edge node monitors collective state and resynthesizes policies when objectives change or element failures occur.

At nanoscale, the BSE operates exclusively as a design-time tool. Because molecular elements run fixed programs encoded in their physical structure, the BSE synthesizes the behavioral program before elements are manufactured rather than downloading it at deployment. The BSE at this scale is a molecular design tool: it accepts collective intents, uses a reaction-diffusion physics planner to determine the chemical emission and receptor parameters that will produce the desired collective behavior, and outputs a molecular design specification rather than executable code. The BSE interface to L7 is unchanged.

3.9 L7 — Choreographer

The Choreographer is the developer-facing surface of Tapestry. Its design philosophy: hide the complexity below, expose an expressive, high-level API, and make it possible for a domain expert with no systems software background to program physical matter. A developer creates a collection of declarative Goals in a Choreo which is provided to the Choreographer which decomposes it into imperative Intents.

Key design decisions at this layer:

- Tapestry SDK: language bindings (Python for research, C for production), with APIs for shape, state, behavior, and sensing.
- Intent API: developers express what they want matter to do, Tapestry figures out how to accomplish it.
- Choreo lifecycle: install, configure, deploy, monitor, terminate. Analogous to Android Activity lifecycle.
- Permissions model: applications declare required physical capabilities (locomotion, bonding, sensing modalities). Elements only grant what the task requires.
- Developer tools: simulator, debugger, profiler, and telemetry dashboard.

The most important observation in Table 3 is the stability of L6 surface and L7 across all three physical scales. The Choreographer API, the Choreo lifecycle model, the Goal-to-Intent translation, the permissions model, and the developer toolchain are identical regardless of the scale of the underlying elements. This stability is a property of the

layered architecture. In the OSI model, swapping the physical medium from copper to fiber does not affect how HTTP applications behave. In Tapestry, swapping the element substrate from ARM silicon to molecular biology does not affect how Choreos behave. This scale-invariance is a structural consequence of the interface contracts that separate each layer from the layers adjacent to it.

4 Illustrative Scenarios

4.1 Factory Spill

Consider a collective of 50 mobile ground robots deployed in an industrial facility to surround and track a chemical spill of unknown and evolving extent. No central coordinator directs the response. Elements must sense the spill locally, self-organize a continuous boundary around it, adapt as the spill spreads and under element failures and battery depletion, and provide real-time visibility to facility operators.

The developer uses the Tapestry SDK to define a Choreo with three Goals:

- **Goal:** Establish and maintain a continuous physical boundary around a detected spill.
- **Goal:** Maintain a minimum element density of one element every two meters along the boundary.
- **Goal:** Set all element warning lights to Red and report boundary geometry to the facility monitoring system.

The Choreographer (L7) knows the capabilities of the collective and translates the Goals into concrete Intents, without specifying the number of elements required:

- **Intent:** `detect_boundary(sensor=chemical_proximity, threshold=0.3ppm)`
- **Intent:** `move_to_boundary_contour(geometry=detected_spill)`
- **Intent:** `distribute_equidistant(spacing=2m, close_gaps=true)`
- **Intent:** `set_indicator(color=WARNING_RED)`
- **Intent:** `report_telemetry(target=facility_monitor, interval=1s)`

These Intents are passed to the Behavior Synthesis Engine (L6) which analyzes these Intents knowing the world model of the collective and synthesizes a behavioral plan for the collective. It immediately encounters a conflict: some elements report battery levels too low to navigate to the spill. They would exhaust their batteries mid-mission, creating gaps in the perimeter at unpredictable locations. The BSE resolves this conflict by preempting the boundary assignment for low-battery elements with an

- **Intent:** `return_to_charging_station()`

and recalculates `distribute_equidistant` for the remaining available elements — stretching their spacing slightly to compensate for the reduced count. The BSE synthesizes local behavioral plans for each element: move toward the nearest detected spill boundary, maintain repulsion from neighboring elements to enforce minimum spacing, and report position and chemical sensor readings to L4 at every gossip cycle.

The Swarm Coordination Runtime (L5) distributes the BSE's plan into localized Tasks assigned to specific elements based on their current positions in the L4 world model:

- **Task** (Healthy Elements): `navigate_to(boundary_sector_A)`
- **Task** (Low-battery Elements): `navigate_to(charging_station)`

As low-battery elements begin their return, the SCR detects through L4 that a 3-meter gap has opened in its sector. Without waiting for BSE replanning, the SCR locally reassigns adjacent elements to expand their spacing and compensate for the gap as best as possible.

- **Task** (Adjacent Elements): `adjust_spacing(low_battery_neighbors, target=2.1m)`

This demonstrates a fast local coordination decision within the SCR's authority that does not require a full BSE replan cycle.

The Collective State Manager (L4) on each element maintains a local world model of its neighbors' positions, chemical sensor readings, battery states, and boundary assignments. As the spill spreads — detected by elements on the inner

edge of the perimeter whose chemical sensors cross the threshold — their position updates propagate through the L4 gossip protocol to all elements within range. Elements on the boundary adjust their positions in response to their neighbors' updated world model entries without any centralized instruction. The collective boundary breathes outward in response to the evolving spill geometry, driven entirely by local world model updates propagating through L4.

If a low-battery element fails before reaching a charging station, its L4 world model entry ages past the stale threshold and is marked expired after 3,000 milliseconds. The L5 SCR detects the quorum change and reassigns tasks to adjacent elements. The facility operator's monitoring system, reading the telemetry stream from L4, sees element status change and tasks reassignment without any operator intervention to successfully contain and report the spill.

4.2 Precision Agriculture

A drone swarm of 200 aerial elements is deployed over a 50-hectare wheat field to identify and treat an outbreak of wheat rust fungus. The developer defines a Choreo with three Goals:

- **Goal:** maintain complete multispectral coverage of the field at 10-meter resolution
- **Goal:** identify regions where spectral signatures indicate fungal infection above a threshold severity
- **Goal:** apply targeted fungicide to infected regions at element-level precision without treating healthy area

The Choreographer binds these Goals into Intents:

- **Intent:** `survey_grid(resolution=10m)`
- **Intent:** `classify_region(pathogen=wheat_rust, threshold=0.7)`
- **Intent:** `apply_treatment(payload=fungicide, region=infected_zones)`

and passes them to the BSE. The BSE faces a planning challenge that the factory spill example does not: the treatment objective cannot begin until the survey objective has produced sufficient field coverage to identify infected regions, creating a temporal dependency between Goals that must be sequenced correctly by the BSE. As survey data accumulates in the L4 world model, the BSE progressively synthesizes treatment routes for elements whose survey sectors are complete, while keeping remaining elements on survey duty. When 15 elements report low payload levels, the BSE reassigns them to a return-to-base Intent and redistributes their treatment sectors among fully-loaded neighbors, all without interrupting the collective's field coverage.

This example exercises L4's sensor fusion at agricultural scale, the BSE's temporal Goal sequencing, and L5's dynamic task reallocation under continuous element state change.

4.3 Surgical Microrobot

A collective of 12 magnetically-actuated microrobots, each approximately 2mm in diameter, is deployed in a patient's vitreous humor to deliver a targeted anti-VEGF drug payload to a retinal lesion identified by preoperative imaging. The developer — in this case a surgical AI system — defines a Choreo with four Goals:

- **Goal:** navigate to within 500 micrometers of the lesion boundary without contacting the retinal surface
- **Goal:** maintain a minimum separation of 300 micrometers between elements to prevent aggregation
- **Goal:** confirm lesion proximity via onboard optical coherence sensing
- **Goal:** release payload simultaneously across all elements on confirmation

The Choreographer operates in CP mode — the safety-critical nature of the deployment means elements freeze rather than act on stale world model data if the mesh partitions. The BSE synthesizes navigation routes using a viscous drag physics model appropriate to the aqueous environment, accounting for the magnetic actuation constraints that limit each element's independent maneuverability. The SCR coordinates the simultaneous payload release across all 12 elements using the BFT consensus mechanism to ensure that a single element's sensor uncertainty cannot trigger premature release.

This example exercises CP mode under safety constraints, the BSE's scale-appropriate physics model, and L5's consensus mechanism for a synchronization decision where partial execution is worse than no execution.

4.4 In-Space Structure Assembly

A collective of 40 autonomous elements is deployed in low Earth orbit to assemble a 10-meter deployable antenna reflector from modular panels that were launched as a compact stack. Ground communication latency of 400 milliseconds makes teleoperation impossible — the collective must operate fully autonomously. The developer defines a Choreo with three Goals:

- **Goal:** self-locate each element relative to the others using inter-element ranging
- **Goal:** assemble into the target reflector geometry within a 90-minute orbital window
- **Goal:** verify structural integrity of all panel bonds before the window closes

The Choreographer operates without any ground feedback loop: the Choreo is uplinked to an edge computer and L7 runs autonomously. The BSE uses an orbital mechanics physics model to account for the differential drag and gravitational gradient forces acting on each element as it maneuvers, planning assembly sequences that exploit orbital mechanics rather than fighting them. When an element fails to confirm a bond after three attempts (detected by L4's world model showing no position change despite actuation commands), the SCR reassigns the bonding task to an adjacent element and replans the remaining assembly sequence around the gap, completing the structure before the orbital window closes.

This example exercises fully autonomous operation without ground infrastructure, the BSE's orbital physics model, and L5's fault recovery under element failure mid-task.

5 Evaluation

The Tapestry architecture has been implemented and validates several results empirically: that world model entries propagate to all elements within bounded time after a state change; that the CP and AP modes degrade differently and predictably under network partition; and that individual element failure and recovery are detected and reflected in collective behavior without centralized intervention. To stress-test Tapestry across different environments, we validate a subset of our results through controlled simulations and the remainder via a real-world collective of robots. All code and scripts required to recreate the results of this section are available at <https://github.com/tapestry-os/>.

5.1 World Model Propagation and CP vs AP Mode Comparison

Simulation Environment. Using Zephyr's `native_sim` simulator, ten Tapestry elements were instantiated as individual processes on a single Linux host, with Unix domain sockets replacing physical radio links for inter-node communication. Each instance executes the full L1–L5 stack with a chosen 100 ms tick rate, 500 ms gossip interval, 1500 ms stale threshold, and 5000 ms expiry threshold. Elements perform a random walk within a bounded 2D space, updating their authoritative position entry at each tick. A Python orchestrator injects partition events by blocking inter-island socket communication for a time and heals them by restoring it, with timestamps logged to millisecond precision. Times are represented as wall time from the orchestrator's perspective. All results presented below are the mean of five independent runs with shaded regions in graphs representing one standard deviation across runs.

Figure 2 shows a controlled partition-and-recovery experiment comparing CP and AP modes under identical conditions: elements are given 1 s to actively gossip and share state information, then 5 s later they are divided into two partitions and 10 s after that the partition is healed. Four metrics are tracked: fraction of elements in degraded state, mean position error between last-seen world model entries and ground truth, mean world model entry age, and minimum inter-element separation.

Partition detection. During partitioning, elements realize they have a stale world model (i.e., a "degraded" state) after the configured 1500 ms time for cross-partition entries to age without receiving gossip. With ten elements in a 5/5 split, each island observes 4 of 9 known peers (≈ 0.44) as fresh, below the configured quorum threshold of 0.5. In CP mode all ten elements immediately freeze their own-state updates, preventing further position divergence. In AP mode elements continue random-walking and updating their own-state entries, taking on a consistency cost in exchange for continued operation. This divergence in behavior between modes is visible in the mean position error panel (Figure 2, second row): AP mode accumulates position error steadily from partition onset, while CP mode holds near zero through the freeze period.

Expiry event. After 5 more seconds, the expiry threshold passes. Cross-partition entries transition from stale to inactive and are removed from the active total. The degraded fraction clears simultaneously for both modes (Figure 2,

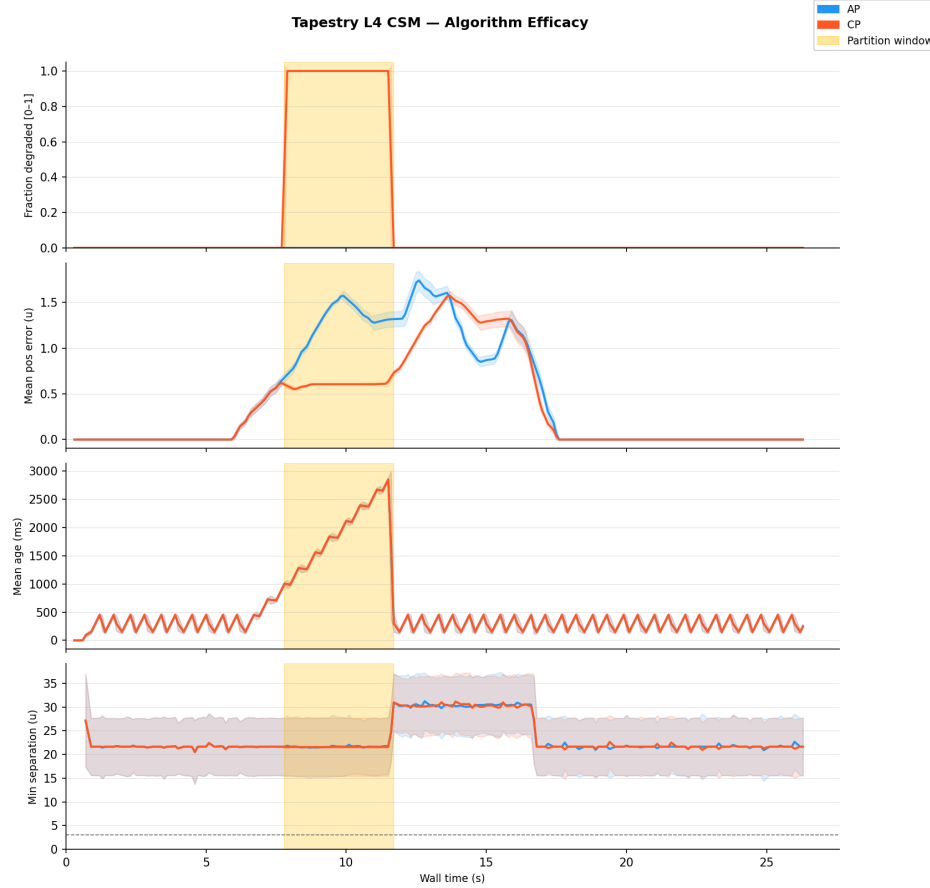


Figure 2: Tapestry L4 CSM simulation showing CP vs AP modes under network partition. Ten elements in a 5/5 split; partition injected at 5 s and healed at 15 s. Yellow shading ends at the expiry event (5 s after degradation) rather than the actual heal. The orchestrator introduces a 1.3 s startup delay before the scenario begins (0.3 s for the broker UDP socket to bind, 1.0 s for element processes to initialize), so all scenario-relative times (wall time) are offset by 1.3 s in the plots. From top: degraded fraction, mean position error, mean world model entry age, and minimum inter-element separation. Shaded bands represent one standard deviation across five independent runs.

top panel). This is not the partition heal: inter-island communication remains blocked. It is an artifact of the expiry mechanism correctly purging entries that are no longer actionable. In CP mode, this quorum restoration triggers simultaneous unfreezing of all ten elements from their pre-partition positions. Elements begin random-walking from their compact cluster, generating a position error buildup corresponding to the elements in the unseen partition. Note this is illustrative: the expiry time and purging rules can be tuned to avoid such divergence as determined by the situation.

The minimum separation panel (Figure 2, bottom row) exhibits a corresponding behavior: when five cross-island entries go inactive at 5 s, the world model shrinks from nine active peers to four. The repulsion equilibrium re-establishes for a five-element island, and fewer elements pushing each other produces a larger equilibrium spacing. Therefore, the minimum separation jumps from approximately 21.6 u to approximately 30 u. This elevated spacing persists until the actual partition heals.

Partition heal. After the 10 second partition is removed, cross-island gossip resumes. AP elements have been in uninterrupted random walk and their within-island world models have been accurate throughout the partition. Reconciliation via Lamport clock merge is fast: position error drops sharply within two gossip cycles. CP elements, having unfrozen only five seconds earlier, present a denser configuration. Both modes recover to zero position error and minimum separation returns to the ten-element equilibrium as cross-island repulsion is restored.

These results validate the CP/AP behavioral distinction asserted in the architecture: CP mode sacrifices availability (element motion halts) to bound consistency divergence during partition; AP mode maintains availability at the cost of accumulated position error (approximately 0.8 u).

5.2 Element Failure and Recovery

Real-world Collective Demonstration. To validate that the L4 world model correctly detects and reflects physical element failure without centralized intervention, a four-element Cutebot ground robot collective was configured running the full Tapestry L1–L5 firmware stack on BBC micro:bit v2 hardware (Nordic nRF52833 processor, BLE 5.0 mesh) [28]. Each element’s NeoPixel indicator LEDs encode its local world model state in real time: green indicates a fresh world model entry, yellow indicates a stale entry approaching the expiry threshold, and red indicates an expired entry or quorum loss.

With all four elements active, each element’s world model maintains fresh entries for all three peers. All indicator LEDs display green. Elements maintain spatial distribution through a simple repulsion mechanism in the L4 world model—no explicit formation command is issued; the distribution emerges from local world model interactions.

Element removal. One element is physically powered off and removed from the operating area to simulate an element wandering farther than the communication link range or dies due to low battery. The remaining three elements detect the absence through the world model aging mechanism: the removed element’s entries age past the stale threshold (1500 ms) and the corresponding peer LED indicators transition from green to yellow on each surviving element. At the expiry threshold (5000 ms), the removed element is marked inactive by the surviving three elements and their repulsion equilibrium recalculates for a three-element collective. No external intervention or reconfiguration command is issued at any point; failure detection and behavioral adaptation emerge entirely from the gossip-based world model update mechanism.

Element return. The powered-off element is reintroduced and powered on. It begins advertising its position via the gossip protocol at its first tick cycle. Within three gossip cycles (of 500 ms each) the world models of all four elements register a fresh entry for the returning element. Indicator LEDs on all four elements return to solid green. The collective’s repulsion equilibrium re-establishes for a four-element collective and elements redistribute to the denser spacing characteristic of the larger population. This physical demonstration of the full failure-detection-and-recovery cycle validates that the L4 consistency mechanism operates correctly on real embedded hardware under real BLE mesh communication conditions, not only in simulation. The indicator LED encoding makes the world model state directly observable to a human viewer in real time, providing an externally verifiable validation artifact that complements the quantitative simulation results.

5.3 Summary

Table 4 summarizes the results of the validation against the architectural specifications. All four claimed behaviors are confirmed: world model convergence within three gossip cycles, CP mode degradation bounding position divergence during partition, AP mode maintaining operational continuity during partition, and physical element failure and recovery detected and reflected within specified thresholds without centralized intervention.

Table 4: Validation outcomes against architectural specifications.

Collective Behavior	Specified	Observed	Platform
Convergence after join	≤ 1500 ms	≤ 1500 ms	native_sim
Stale detection	≤ 1500 ms	≤ 1500 ms	Cutebot
Expiry after failure	≤ 5000 ms	≤ 5000 ms	Cutebot
Post-return convergence	≤ 1500 ms	≤ 1500 ms	Cutebot
CP bounds divergence	< AP during partition	$\approx 0.8 u$ at heal	native_sim
AP maintains continuity	No freeze in AP	Confirmed	native_sim

6 Open Standard Strategy

Tapestry is released as open source under the Apache 2.0 license. This open licensing structure lowers adoption barriers, enabling hardware vendors to independently manufacture and ship Tapestry-compatible elements free from

permissions, proprietary negotiations, or royalty obligations. The patent grant ensures that contributors cannot later assert the patents covering their contributions against the platform’s users. Furthermore, there are no copyleft obligations on products built on Tapestry. Any company building a proprietary product on top of Tapestry is not required to open source their development.

As an open platform, Tapestry fosters a compounding ecosystem of community benefits. It systematically aggregates hardware compatibility as organizations contribute upstream L1 drivers. Simultaneously, continuous security audits, academic formal verification, and widespread adversarial testing drastically minimize defect density in the core codebase. Tapestry also serves as an open testbed where the distributed systems research community can evaluate, challenge, and extend novel protocols. Researchers across disciplines can contribute to the problems most relevant to their expertise, with the shared platform accumulating results across independent efforts. Ultimately, this open model assures Choreo and tool developers that efforts will remain reusable across an expanding matrix of hardware platforms.

The initial release of Tapestry fully implements L1 through L5, with L6 and L7 provided as open specifications with functional stub implementations. The L1-L2 runtime is built on the Zephyr RTOS, leveraging its priority-based preemptive scheduling, deterministic interrupt latency, native power management, and a hardware abstraction framework that maps directly to L1’s devicetree model. This implementation has both Zephyr’s `native_sim` (on both ARM and x86) and select robotics hardware targets.

Moving up the stack, the release includes a complete implementation of L3-L5. Notably, this incorporates the L3 gossip protocol and mesh routing layer, L4 features with CP/AP modes and Lamport clock reconciliation, and L5 coordination runtime equipped with lightweight peer-filtering BFT mitigation, message-free deterministic role election, ordinal peer-rank indexing for L6 task decomposition, and quorum-loss detection with abort signaling. Together, this constitutes a complete, runnable, and testable coordination stack that any research group or hardware manufacturer can deploy on existing systems today.

The L6 and L7 stubs define the interface contracts — the API surface that Choreos target and that BSE implementations must satisfy — without implementing the physics-aware planner, emergent behavior synthesis, or ML inference runtime. The stubs are functional in simulation: a developer can write a Choreo, observe the Goal-to-Intent translation, and see simplified behavioral programs distributed to elements. The stubs make the API surface concrete and testable, so the hardest research problems (see Section 7) are open for community contributions.

7 Open Research Challenges

The nine challenges identified in this section represent the frontier where Tapestry’s architectural claims meet unsolved problems in distributed systems, control theory, materials science, and synthetic biology. Each challenge is a prerequisite for production deployment in at least one of the commercial verticals identified in Table 1, and each is an open research problem in its own right — independent of the broader Tapestry platform.

These challenges are presented in dependency order: foundational solutions establish the telemetry and resource bounds required to unlock progress on the ones that follow. Some (1, 5, and 6) are close to being solved in adjacent domains, such that focused engineering work can bring them to fruition. Some (3, 4, and 7) are active research areas where Tapestry’s specific requirements provide novel problem formulations that build on established foundations. The remaining ones (2, 8, and 9) are genuinely open research problems at the intersection of multiple disciplines, requiring the kind of long-horizon collaborative effort that an open platform and an active research community are best positioned to sustain. One benefit of Tapestry’s open source release is to provide a way for the scientific community to design, develop, and validate solutions to these challenges.

1. The Physical CAP Theorem. As discussed in Section 2.1, the autonomy of physical matter introduces temporal validity decay—a dimension absent from classical distributed systems theory. While Tapestry’s L4 CSM provides empirical consistency modes to mitigate this decay, a rigorous, formal framework characterizing the three-way trade-off between consistency, availability, and temporal validity remains an open theoretical challenge. Future research must formalize these staleness functions based on substrate dynamics: e.g., the validity decay rate in a fast-moving aerial drone is far faster than that of a stationary structural actuator. Developing this mathematical model is essential to parameterize dynamic quorum threshold calculations, bound state divergence during network disruptions, and define deterministic reconciliation semantics for post-partition physical merges.

2. Scale-Invariant Hardware Abstraction. The HAL in conventional operating systems assumes a Von Neumann architecture which breaks down at nanoscale. The open problem is a hardware abstraction model that remains coherent across six orders of magnitude of physical scale, from ARM microcontrollers to biochemical state machines, without requiring different application code above L2. A molecular machine does not fire an interrupt when a ligand binds: it

transitions to a bound state with a certain probability per unit time [27]. The HAL must expose this stochastic event model in a form that the deterministic L2 Runtime can program against.

3. The Inverse Problem of Emergence and Behavioral Synthesis. As highlighted in Section 2.2, translating a macroscale collective target into local, per-element programs represents the modern evolution of the ensemble programming challenge. While predicting collective outcomes from known local rules is computationally tractable, solving this inverse problem of emergence is notoriously complex. The challenge is structurally analogous to the inverse protein folding problem in biochemistry. Much like AlphaFold’s data-driven breakthrough in predicting three-dimensional structures, utilizing deep machine learning over vast, forward-simulated datasets represents the most promising avenue for application within L6. The unresolved theoretical and engineering challenge is developing a generalized synthesis framework capable of generating novel behavioral programs that are both provably verifiable against local physical constraints and executable within the strict latency budgets of real-time distributed feedback loops.

4. Cross-Scale Consensus Under Heterogeneous Communication. As established in Section 2.3, operating a consensus engine under conditions that violate independent failure modes and bounded execution windows remains an open challenge. While the L5 SCR provides an empirical framework to transition across execution regimes, the challenge is creating a consensus protocol variant for physically embodied collectives that: handles heterogeneous communication latency without blocking consensus rounds awaiting high-latency participants, supports continuous membership churn without global reconfiguration rounds, and maintains correct behavior under spatially correlated failure modes (e.g., a drone crash simultaneously eliminating its neighbors’ communication). The bisimulation equivalence between the BFT, stigmergic, and reaction-diffusion coordination regimes asserted in the L5 architecture also requires formal proof.

5. Distributed Spatial Indexing Under Continuous Motion. The L4 world model must support efficient nearest-neighbor queries — which elements are within gossip radius, which elements are within collision distance — across a collective where every element’s position is continuously changing and the index is itself distributed across the elements with no central coordinator. Centralized spatial indexes including R-trees, k-d trees, and octrees are well-understood and efficient, but require a central node to maintain the index. Distributed hash tables provide index structures without a central coordinator but are designed for static key-value data rather than continuously-evolving spatial positions. The open problem is a distributed spatial index data structure that maintains approximate nearest-neighbor query capability for large N across a continuously-moving element collective, with bounded staleness guarantees, without a central coordinator, and within the memory constraints of resource-constrained embedded processors.

6. Power-Aware Collective Scheduling. Conventional operating systems schedule computation to optimize throughput or latency within a fixed power budget. Tapestry must schedule collective behavior to optimize against the collective energy budget, distributed unevenly across elements with heterogeneous and time-varying individual energy states. Assignment of locomotion tasks changes the swarm topology, which changes who can communicate with whom, which changes the optimal assignment (NP-hard). At nanoscale the problem compounds further: elements harvesting energy from chemical gradients or thermal differentials have energy budgets that fluctuate with local environmental conditions rather than depleting monotonically [27], requiring a scheduling model that accommodates stochastic energy income rather than only deterministic energy expenditure. The open problem is a tractable distributed approximation algorithm for collective power-aware task scheduling that: runs using only local information available to each element, produces assignments within a provable bound of the globally optimal assignment, and recomputes efficiently as individual energy states and collective topology change. The algorithm must generalize across energy models from batteries and ambient RF harvesting at macroscale to chemical gradient harvesting at nanoscale.

7. Physics-Aware Planning at Scale. The L6 BSE’s physics-aware planner must translate collective behavioral objectives into element actuation sequences under physical constraints. Existing ensemble path planning solutions in multi-robot systems incorporating conflict-based search scale to tens of robots in discrete environments [29] but not to thousands of elements in continuous physical space with scale-dependent physics. The open problem is a hierarchical planner architecture that: decomposes ensemble planning into tractable subproblems without losing the inter-element coupling that makes collective behavior physically meaningful, is parameterizable over physics models that differ in kind rather than only in constants, and integrates a closed-loop feedback controller that corrects deviations from the intended trajectory in real time rather than requiring replanning from scratch. The FEA simulation interface identified in the L6 architecture addresses the continuum material case; the general multi-scale planning problem for discrete element collectives remains open.

8. Formal Verification of Emergent Collective Behavior. For Tapestry to be deployable in regulated domains (surgical microrobots, morphing aircraft structures, exoskeletons, etc.), the collective behaviors produced by a given set of local rules must be formally verifiable: proven to remain within specified behavioral bounds under all conditions, not merely tested to work under sampled conditions. Swarm behavior operates over continuous physical state spaces, produces emergent behaviors that are by definition not explicitly programmed, and exhibits sensitivity to initial conditions that makes exhaustive verification intractable. Lyapunov stability analysis provides formal stability guarantees for continuous dynamical systems but has not been extended to the distributed, decentralized, emergent behavior setting of large element collectives. The open challenge is a formal verification framework for emergent collective behaviors that provides provable behavioral bounds (e.g., "this collective will converge to the target formation within t seconds with probability $> 1 - \epsilon$ ") without requiring exhaustive simulation of the continuous physical state space.

9. Biochemical Substrate Interface. At nanoscale, the computational paradigm underlying Tapestry's L1 and L2 is expressed through molecular geometry. While DNA strand displacement circuits [23] have demonstrated universal computation in molecular substrates, and chemical reaction network theory provides a mathematical framework for reasoning about molecular behavior, the challenge is providing the software interface bridge that maps this stochastic, continuous-time reaction kinetics of molecular machines onto a deterministic, discrete-time, instruction-based model. This formal computational model must be expressive enough to implement the gossip protocol message format and world model data structure in terms of molecular state transitions while abstracting the stochastic physical details from L3 and above. This is the longest-horizon challenge in the architecture and the one most requiring collaboration between the systems software and synthetic biology communities.

8 Conclusion

Physically reconfigurable matter systems, such as aerial swarms, warehouse fleets, surgical microrobots, and morphing structures, share a fundamental coordination challenge that is currently solved via isolated, domain-specific software stacks. This fragmentation stifles progress, forcing developers to rebuild core infrastructure for every deployment and preventing the accumulation of shared capabilities across domains.

Tapestry addresses this gap by introducing a seven-layer operating system architecture that establishes a stable, hardware-agnostic boundary between physical elements and collective application logic. Intentionally scale-invariant, the architecture provides a consistent developer surface whether executing on modern silicon microcontrollers or long-horizon nanoscale molecular machines, requiring no central coordinator at any layer. The Tapestry reference implementation is released as open source under the Apache 2.0 license at <https://github.com/tapestry-os/>, offering a concrete, runnable platform to empirically validate and advance solutions to the nine cross-disciplinary research challenges of kinetic consistency, stochastic power scheduling, and emergent behavior synthesis. Ultimately, Tapestry transforms the landscape of reconfigurable matter from a collection of isolated, bespoke implementations into a compounding, unified ecosystem of portable, scalable software capabilities.

Acknowledgements

This work was conducted independently and is not affiliated with any institution. The author utilized Claude (Anthropic) for code prototyping and manuscript refinement, and Gemini (Google) for additional writing support. All architectural designs, technical decisions, and final code reviews were performed by the author, including manual code hardening and verification on actual hardware to ensure the stability and integrity of the resulting system.

References

- [1] Rubenstein, M., Cornejo, A., and Nagpal, R. (2014). Programmable self-assembly in a thousand-robot swarm. *Science*, **345**(6198), 795–799. DOI: 10.1126/science.1254295.
- [2] Jessen, J. (2024). "Drones 'Game Changing' for Aramco, Shell, Chevron and BP." *Energy Digital*, Oct 19, 2024.
- [3] Greenawalt, T. (2025). "Amazon has more than 1 million robots that sort, lift, and carry packages — see them in action." *Amazon News*, Oct 22, 2025.
- [4] Strzelczyk, P., Tomczewski, K., and Wróbel, K. (2022). The Middleware for an Exoskeleton Assisting Upper Limb Movement. *Sensors*, **22**(8), 2986. DOI: 10.3390/s22082986
- [5] Belton, B., Baldiga, L., Justice, S., Minten, B., Narayanan, S., and Reardon, T. (2025). Can the global drone revolution make agriculture more sustainable? *Science*, **389**(6758), 972–976. DOI: 10.1126/science.ady1791

- [6] Parancheerivilakkathil, M. S., Pilakkadan, J. S., and Ajaj, R. M. (2024). A review of control strategies used for morphing aircraft applications. *Chinese Journal of Aeronautics*, **37**(4), 436–463. DOI: 10.1016/j.cja.2023.10.010
- [7] Jacobson, S. (2012). NASA Dryden Status: ACTE Flight Preparations. *Air Charter Guide Steering Committee Meeting 110*. NASA Technical Reports Server.
- [8] Lomas, N. (2016). “MIT creates a control algorithm for drone swarms.” *TechCrunch*, Apr 22, 2016.
- [9] Landers, F. C., et al. (2025). Clinically ready magnetic microrobots for targeted therapies. *Science*, **390**(6770), 710–715. DOI: 10.1126/science.adx1708
- [10] Song, T. and Qian, L. (2025). Heat-rechargeable computation in DNA logic circuits and neural networks. *Nature*, **646**, 315–322. DOI: 10.1038/s41586-025-09570-2
- [11] Brewer, E. (2000). Towards robust distributed systems. *Proceedings of the Annual ACM Symposium on Principles of Distributed Computing (PODC)*, p. 7. DOI: 10.1145/343477.343502.
- [12] Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. (2011). Conflict-free replicated data types. In *Proceedings of the 13th International Conference on Stabilization, Safety, and Security of Distributed Systems (SSS'11)*, Springer-Verlag, Berlin, Heidelberg, pp. 386–400.
- [13] Garcia-Molina, H. and Salem, K. (1992). Main memory database systems: An overview. *IEEE Transactions on Knowledge and Data Engineering*, **4**(6), pp. 509–516.
- [14] Ongaro, D. and Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, USENIX Association, USA, pp. 305–320.
- [15] Castro, M. and Liskov, B. (1999). Practical Byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI '99)*, USENIX Association, USA, pp. 173–186.
- [16] Fall, K. (2003). A delay-tolerant network architecture for challenged internets. In *Proceedings of the 2003 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM '03)*, ACM, pp. 27–34. DOI: 10.1145/863955.863960
- [17] Pinciroli, C. and Beltrame, G. (2016). Buzz: An extensible programming language for heterogeneous swarm robotics. In *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE Press, pp. 3794–3800. <https://doi.org/10.1109/IROS.2016.7759558>
- [18] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., and Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, **7**(66). DOI: 10.1126/scirobotics.abm6074.
- [19] Saridis, G. N. (1983). Intelligent robotic control. *IEEE Transactions on Automatic Control*, **28**(5), pp. 547–557.
- [20] Goldstein, S. C. and Mowry, T. C. (2004). Claytronics: A Scalable Basis For Future Robots. *RoboSphere*, 2004.
- [21] Warneke, B., Last, M., Liebowitz, B., and Pister, K. S. J. (2001). Smart Dust: Communicating with a Cubic-Millimeter Computer. *Computer*, **34**(1), pp. 44–51.
- [22] Seeman, N. (1982). Nucleic acid junctions and lattices. *Journal of Theoretical Biology*, **99**(2), pp. 237–247.
- [23] Qian, L. and Winfree, E. (2011). Scaling Up Digital Circuit Computation with DNA Strand Displacement Cascades. *Science*, **332**(6034), pp. 1196–1201. DOI: 10.1126/science.1200520
- [24] ISO/IEC 7498-1:1994. Information technology — Open Systems Interconnection — Basic Reference Model.
- [25] Thorton, S. (2018). “Zephyr: An open-source RTOS for IoT.” *EE World*, Sept 24, 2018.
- [26] Sliwa, B., Schmutte, M., Pillmann, J., and Wietfeld, C. (2019). Performance Evaluation and Optimization of B.A.T.M.A.N. V Routing for Aerial and Ground-Based Mobile Ad-Hoc Networks. In *Proceedings of the IEEE 89th Vehicular Technology Conference (VTC-Spring)*, pp. 1–7. DOI: 10.1109/VTCSpring.2019.8746361.
- [27] Akyildiz, I. F., Brunetti, F., and Blázquez, C. (2008). Nanonetworks: A new communication paradigm. *Computer Networks*, **52**(12), pp. 2260–2279.
- [28] Demonstration video at <https://www.youtube.com/@tapestry-os>
- [29] Sharon, G., Stern, R., Felner, A., and Sturtevant, N. R. (2015). Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, **219**, pp. 40–66. DOI: 10.1016/j.artint.2014.11.006